

DDIA Lab 02 Report

Team Members

Name	ID
<u>عمر هانی ابراهيم بشر</u>	<u>21010891</u>
<u>محمد حسن محمد قبّاري حسن عبد الجواد</u>	<u>21011115</u>
<u>محمد محمد ابراهيم علي حسن</u>	<u>21011211</u>
<u>نيره ابراهيم احمد محمد الخياط</u>	<u>21011504</u>

[Github repo](#)

- MySQL schema

```
CREATE TABLE ratings ( user_id VARCHAR(10),  
    -- User ID as a string movie_id VARCHAR(10),  
    -- Movie ID as a string rating INT CHECK (rating BETWEEN 1 AND 10),  
    -- Rating (1-10) PRIMARY KEY (user_id, movie_id) -- Composite Primary Key );
```

- MongoDB schema

```
{  
  "collection": "movies",  
  "fields": {  
    "_id": {  
      "type": "ObjectId",  
      "description": "Unique identifier for the document (auto-generated by MongoDB)"  
    },  
    "movieId": {  
      "type": "string",
```

```
    "description": "Unique identifier for the movie",
    "required": true
  },
  "name": {
    "type": "string",
    "description": "Name of the movie",
    "required": true
  },
  "description": {
    "type": "string",
    "description": "Description of the movie",
    "required": false
  }
},
"indexes": [
  {
    "keys": {
      "movieId": 1
    },
    "options": {
      "unique": true
    }
  }
]
```

Part a

Performance -> 3000 thread , ramp –up time = 1 sec , loop count = 1.

90% -> 5.5 sec

95% -> 5.9 sec

99 -> 6.4 sec

Througput -> 415.1 request /sec

7000 -> upper bound on requests , 7000 thread , ramp up time = 1 sec , loop count = 1

8000 -> break with error.

- Proof of caching benefit (median and P90 Latency and throughput before and after)

1000 thread with loop count = 100 and ramp up time = 100 sec.

Before with 100k request :

Average : 8856 msec.

Median : 5605 msec.

P90 : 19669 msec

Throughpt : 81 request/sec

After with 100k request :

Average : 2 msec.

Median : 1 msec.

P90 : 6 msec

Throughpt : 1000 request/sec

After cache :

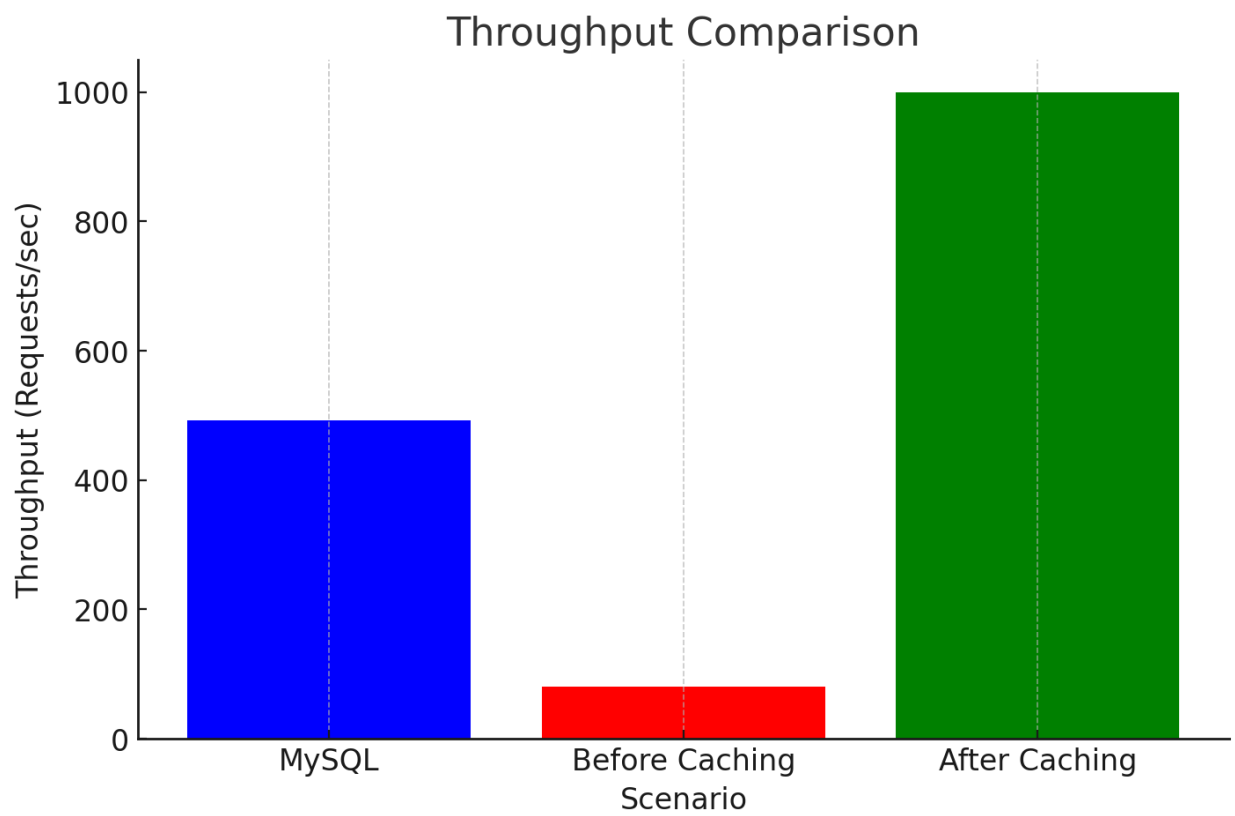
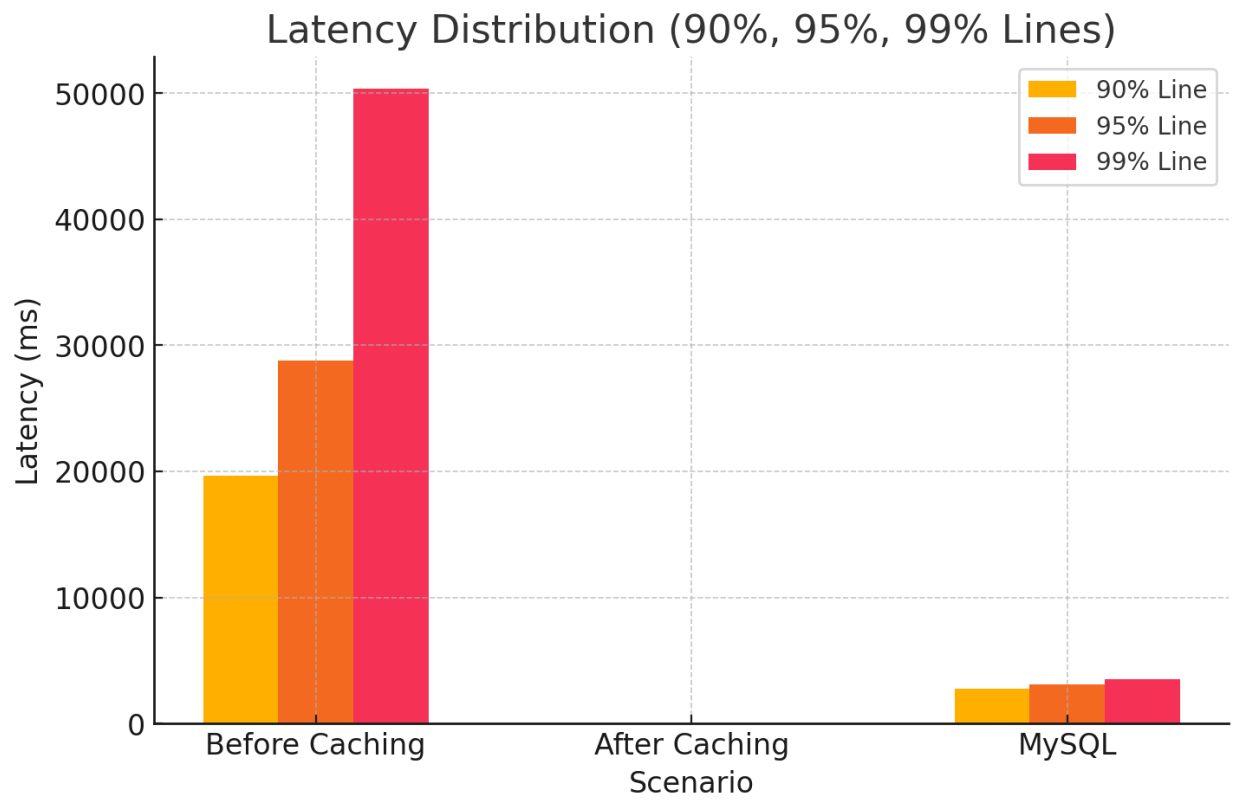
Stress test -> 5000 request -> error

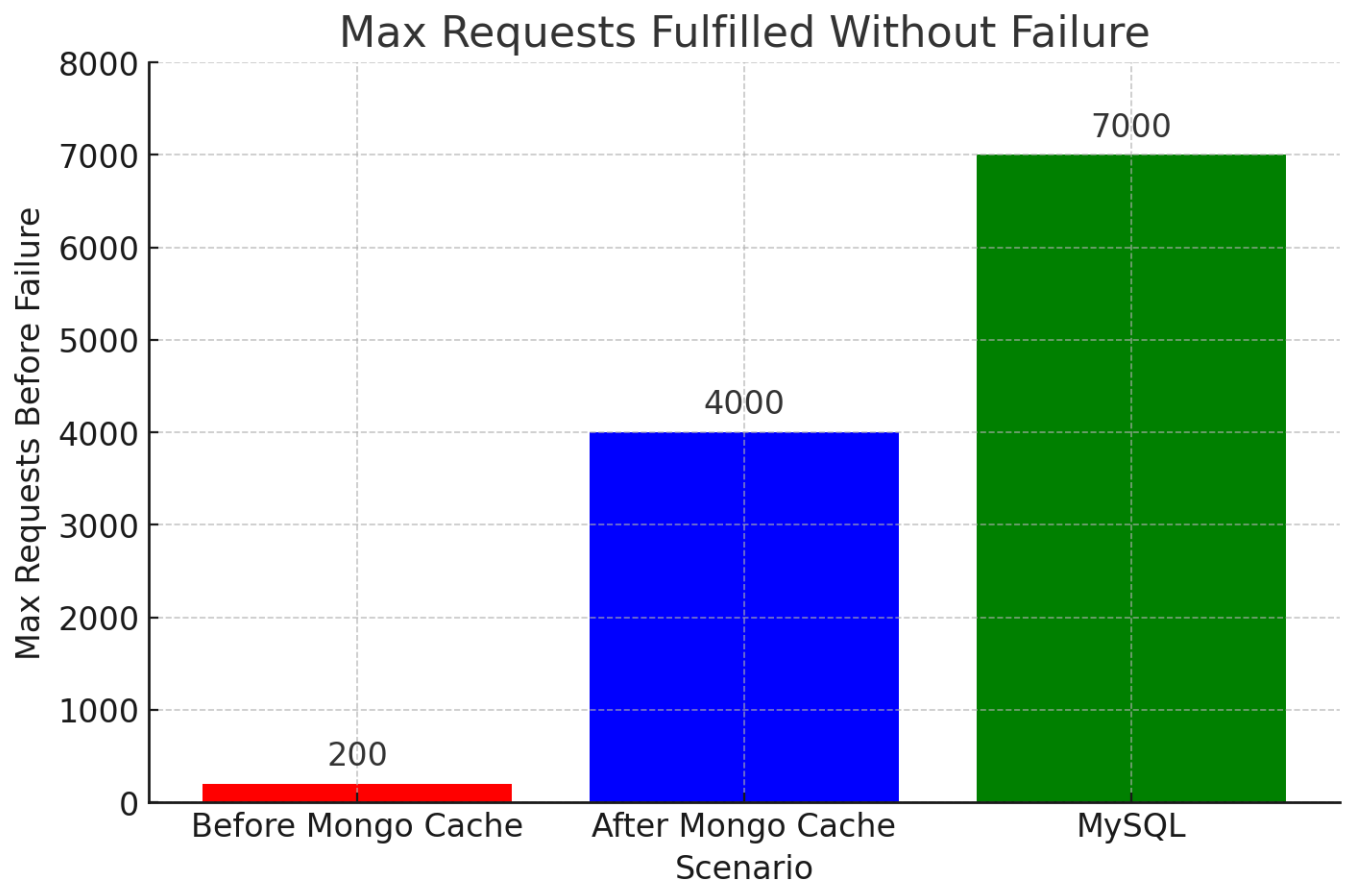
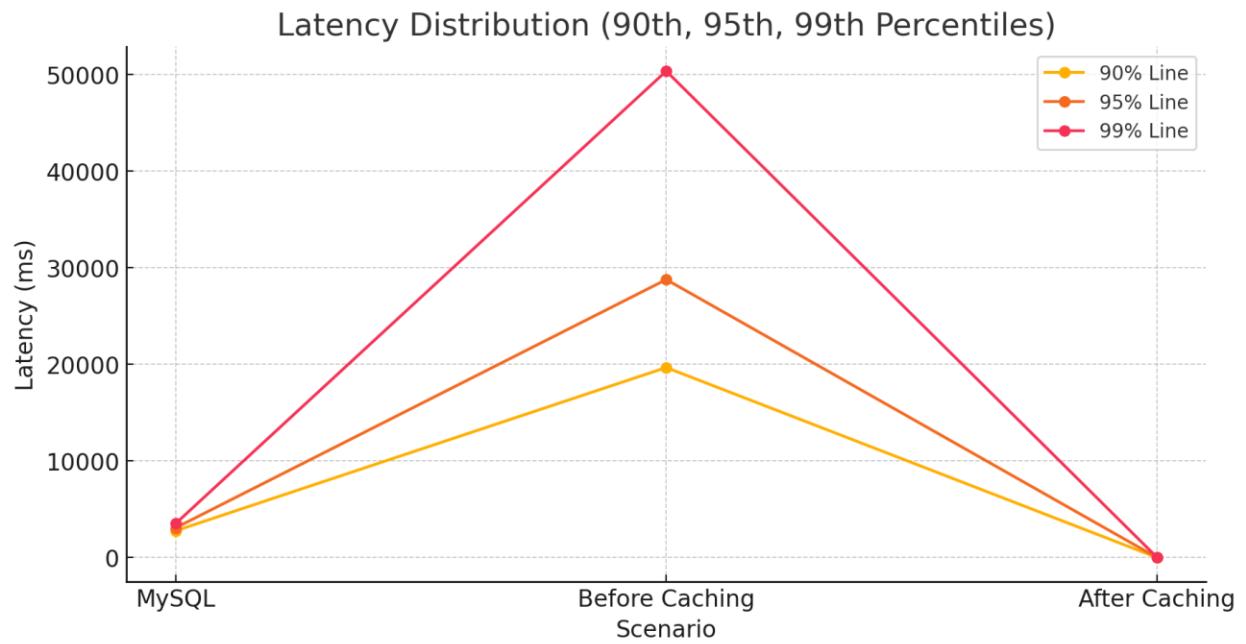
Performance -> 3000 thread

p90 -> 1321 msec

throughput -> 930 request/second

- A running version of the new application
- JMeter Test Plans (jmx files for both performance and stress tests)
- JMeter measurements in any chart you see fit (line, column, etc,) and conclusion which includes:
 - Latency distribution under normal load
 - Throughput
 - Maximum number of requests that your server fulfill without failure





- Answers to all discussion questions

1. Why did we recommend a relational DB (MySQL) for this service?

MySQL is a good choice because:

- **Consistent & Reliable:** Ratings need **accurate transactions** (ACID properties).
- **Good for Relationships:** It handles **user-movie-rating links** well.
- **Fast Queries:** MySQL allows **quick lookups** using indexes.

2. Why did we suggest caching in this service, not other services?

Caching helps the **MovieDB Service** because:

- **API Calls Are Slow:** Fetching from MovieDB takes time & costs resources.
- **Data Doesn't Change Often:** Movie details rarely change, so caching works well.
- **Big Performance Boost:** Instead of waiting for an API call, results come **instantly** from MongoDB.
- **Why not other services?**
 - **Ratings Service** → Ratings change **too often**, making caching unreliable.
 - **User Service** → User data is **sensitive** and must always be fresh.

3. From which data source does the Trending Movies Service fetch its data? Is it adequate? How can it be improved?

- **Data Source** → It gets ratings from **MySQL (Ratings Service)**.
- **Is it good?**
 - **Yes**, if we **precompute** the top 10 movies instead of calculating every time.
 - **No**, if we fetch & sort **millions of ratings on demand** (too slow).
- **How to improve?**
 - **Precompute Rankings** → Run a background job every few minutes.
 - **Use Fast Storage** → Save results in **Redis or MongoDB** for quick access.
 - **Optimize MySQL** → Add **indexes** to speed up queries.